
E101 Final Project: Analog and Digital Filters

By

Jared Carreño

Emma Angel

Adne Brako

Katrina Nguyen

I. Background

This E101 project focused on designing, building, and testing a low-pass filter system to generate a 10 Hz sinusoidal signal from a 10 Hz pulse train. To accomplish this task, a pulse train was generated using an oscillator circuit and an Arduino IDE microprocessor where the pulse train was connected to the low pass filter using the Sallen key 4th order Butterworth filter. The output of the butterworth filter was then passed through a low pass digital comb filter (using arduino) which was modified to produce a sinusoidal signal (10 Hz). The Fourier series coefficients of the input and output of each filter were analyzed and measured, and the theoretical and the actual data were compared.

A two stage Sallen Key circuit schematic as well as the Butterworth equation were key to the project. Equation 1 represents the transfer function of the low pass Butterworth Filter, where $H(s)$ is the frequency response function, and N is the order of the filter and ω_c is the corner frequency.

$$H(s)H(-s) = \frac{1}{1 + \left(\frac{s}{j\omega_c}\right)^{2N}} \quad (\text{Eq.1})$$

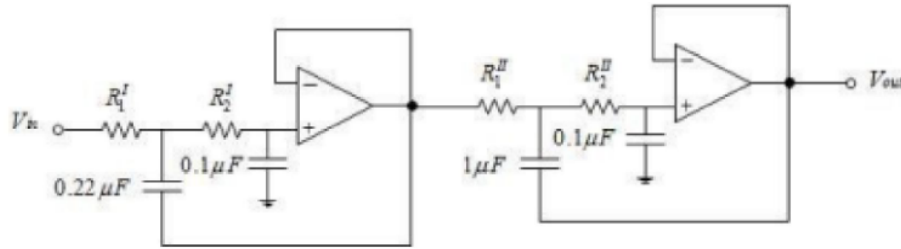


Figure 1: Two stage Sallen Key circuit

The 4th order Butterworth Filter was implemented using the two stage Sallen Key circuit in Figure 1. The resistor values for the circuit were calculated and the closest standard resistors were chosen, as shown in Table 1:

Resistor	Calculated Values	Standard Values
R_1	46506.46 Ω	47 k Ω
R_2	247543.31 Ω	270 k Ω
R_3	26605 Ω	27 k Ω
R_4	95206 Ω	100 k Ω

Table 1: Calculated Resistor Values With the Corresponding Standard Resistor Values

The transfer function in Equation 2 was for the analog Butterworth filter that was used to solve for the resistor values. The purpose of this Butterworth filter was to remove higher frequency components of the signal.

$$H(s) = \frac{1}{s^2 R_1 R_2 C_1 C_2 + s(R_1 + R_2)C_2 + 1} \quad (\text{Eq. 2})$$

The transfer function of the comb filter, as seen from Equation 3, was used to design the digital filter. Where M is the number of zeros and b_0 is the scaling factor.

$$H_{comb}(z) = b_0(1 - z^{-M}) \quad (\text{Eq. 3})$$

The comb filter can be adjusted by eliminating conjugate pairs of zeros to form passbands at specific frequencies while leaving stop bands at the other frequencies.

II. Experimental Results

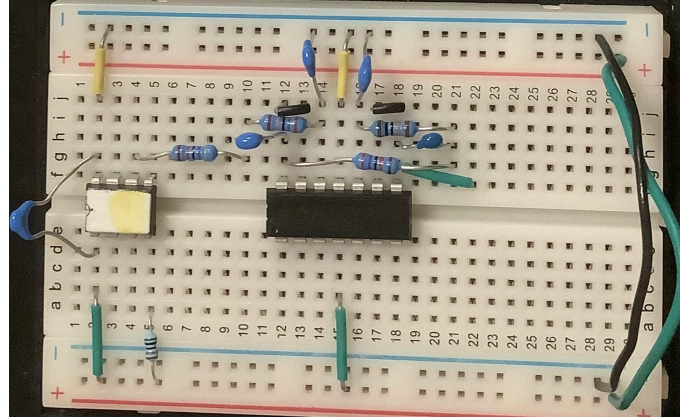


Figure 2: Pulse train PIC controller feeding into sallen key filter

Figure 2 shows the circuit implementation of the designed system. It's composed of an 8 pin PIC microcontroller which generates a pulse wave that is fed into the Sallen-Key filter. The analog filter is implemented with two op amps in series. Data is taken from the output of the pulse wave which serves as the input to the analog filter and the output of the analog filter. The data was measured using oscilloscope probes to visualize the live outputs and collected using the pinouts on an Arduino microcontroller.

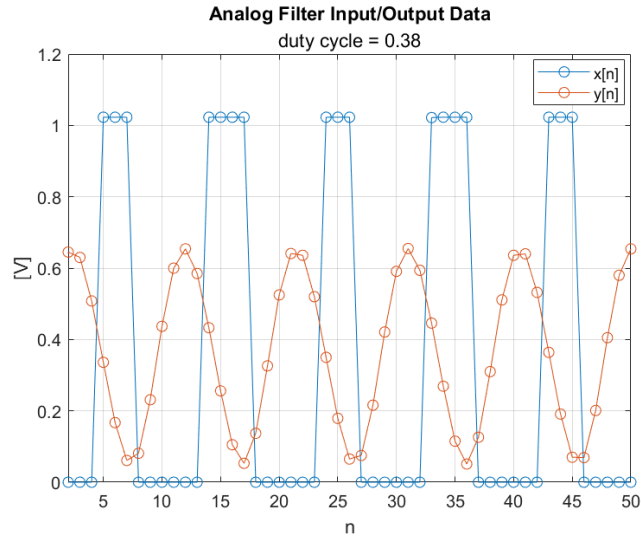


Figure 3: Input and output of the Analog Filter

Shown in Figure 3 is the input and output data from the salient key analog filter sampled at 100 Hz. The data is shown against the sampling number n . The input $x[n]$ is a 0 V to 1 V pulse wave generated from an 8 pin PIC microcontroller with a duty cycle of approximately 0.38. The output $y[n]$ is a sinusoidal signal with a peak to peak voltage of approximately 0.6 V. The resulting output $y[n]$ is delayed and has a decreased amplitude compared to the input pulse train which exemplifies the analog filters' low-pass characteristics. The data was sampled with an Arduino Microcontroller using the code in Appendix B.

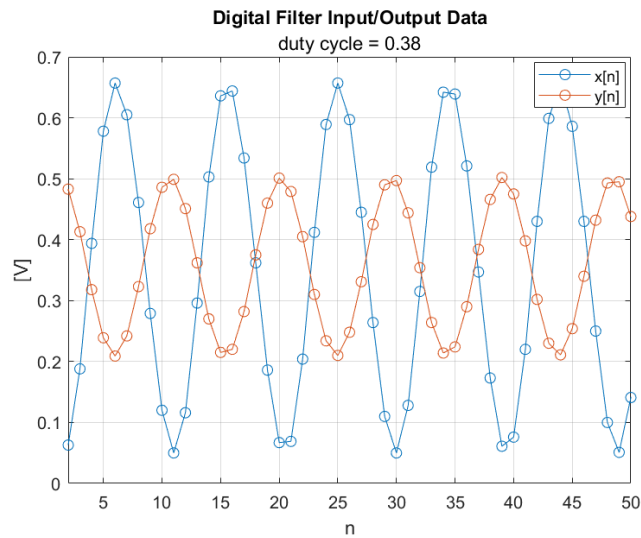


Figure 4: Input and output of the Digital Filter

Shown in Figure 4 is the input and output data from the digital modified comb filter sampled at 100 Hz. The input $x[n]$ is the 0.6 V sinusoidal signal from the output of the analog filter shown in Figure 3. The output $y[n]$ is a sinusoidal signal with a peak to peak voltage of approximately 0.3 V. The resulting output $y[n]$ is delayed and has a decreased amplitude compared to the input

pulse train which exemplifies the digital filters' low-pass characteristics. The data was sampled with an Arduino Microcontroller using the code in Appendix A.

III. Analysis of Results

Using the Data Processing code from Appendix A, the input and output data from the samples signals were extracted from the Arduino files. To estimate the Fourier series coefficients of the sampled pulse train, the following equation was use:

$$|a_k| = \left| \frac{A \sin(k\pi d)}{N \sin(k\pi/N)} \right| \quad (\text{Eq. 4})$$

where $A = 1023$ is the amplitude of the pulse train, as shown in Figure 3, k is the harmonic number, $d = 0.38$ is the duty cycle, and $N = 10$ is the number of samples in one period. The coefficient values were calculated for the harmonics $k = 1, 2, 3, 4$ and listed in the table below:

k	$ a_{k, \text{estimated}} $	$ a_{k, \text{measured}} $	$ a_{k, \text{hann}} $
1	293.6877	271.607	141.205
2	142.5116	100.899	59.6946
3	16.1609	18.7723	9.70911
4	101.1190	71.3632	37.4506

Table 2: Pulse Train Estimated and Measured Fourier Coefficients

Using the FFT code and the Hanning Window FFT Code in Appendix A, the discrete Fourier transform of the pulse train was taken. The Fourier series coefficients were then measured from Figure 5 and recorded in Table 2:

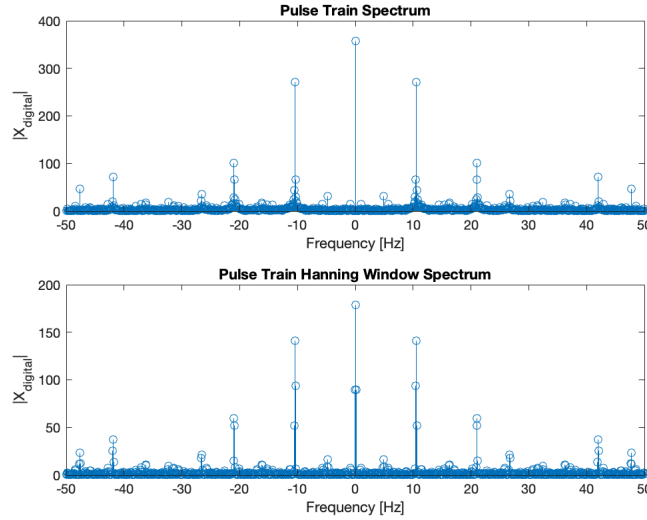


Figure 5: Pulse Train Spectrum and Hanning Window Spectrum

Estimating the Fourier series for the input and output of the analog filter were done in a similar

manner as the pulse train. Since the input of the analog filter was the pulse train itself, the measured $|a_{k,hann}|$ values were carried over into Table 3.

k	$ a_{k,hann} $	$ H(e^{j\hat{w}}) $	$ b_{k,estimated} $	$ b_{k,measured} $	$\frac{ b_{k,estimated} }{ a_{k,hann} }$	$\frac{ b_{k,measured} }{ a_{k,hann} }$
1	141.205	0.63	88.95	72.9	0.25	0.52
2	59.6946	0.054	3.22	2.78	0.040	0.047
3	9.70911	0.01	0.097	0.316	0.025	0.033
4	37.4506	0.0028	0.105	.08	0.020	0.020

Table 3: Analog Filter Estimated and Measured Fourier Coefficients

The FRF of the analog filter was then plotted using Equation 5 and the Sallen Key FRF code (Appendix A):

$$H_{overall}(s) = \frac{1}{s^2 R_1 R_2 C_1 C_2 + s(R_1 + R_2)C_2 + 1} * \frac{1}{s^2 R_3 R_4 C_3 C_4 + s(R_3 + R_4)C_4 + 1} \quad (\text{Eq. 5})$$

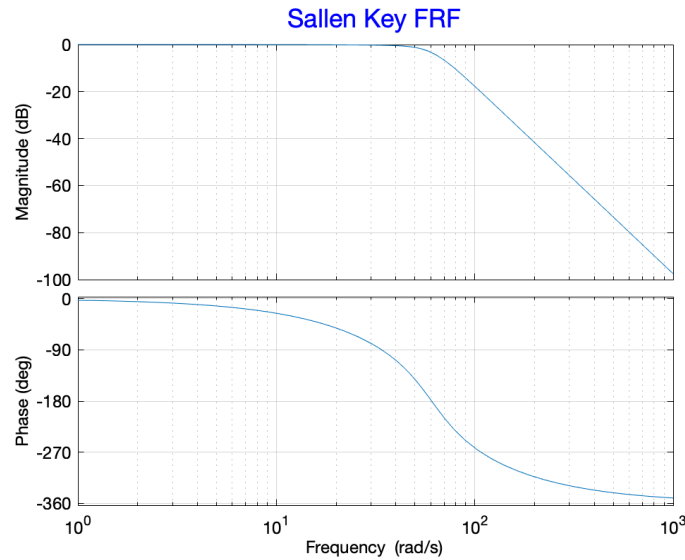


Figure 6: Bode Plot of the Analog Filter

The log magnitudes at the frequencies of the first four harmonics (10 Hz = 62.8 rad/s, 20 Hz = 125.6 rad/s, 30 Hz = 188.5 rad/s, and 40 Hz = 263.8 rad/s) were extracted from Figure 6 and converted to the linear space magnitude with Equation 6.

$$|H(e^{j\hat{w}})| = 10^{\frac{\log \text{magnitude}}{20}} \quad (\text{Eq. 6})$$

The values were then multiplied by the corresponding input coefficient, resulting in the estimated

values for the output coefficients. The measured values of the Fourier series coefficients for the output of the analog filter were then found from the FFT in Figure 7, and the ratio between the output and input values were calculated for the coefficients and recorded in Table 3.

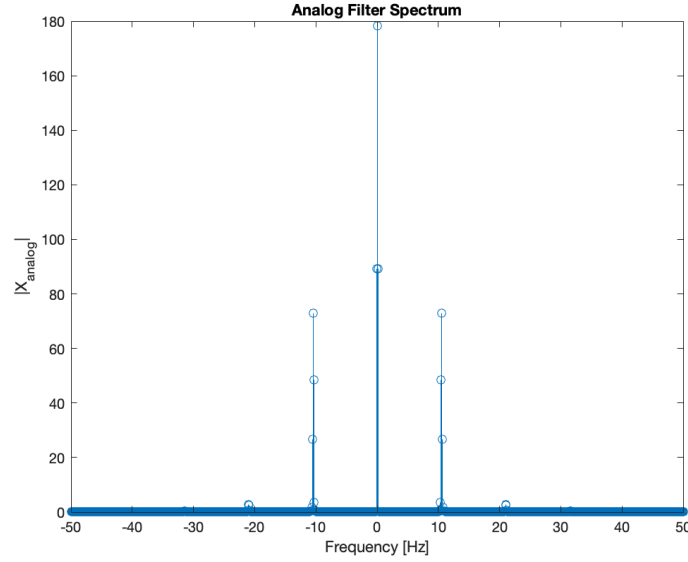


Figure 7: Hanning Window Fourier Spectrum of the Analog Filter

For the digital filter the coefficients for the input, $|b_{k,measured}|$, were carried over from the analog filter. The output coefficients were estimated from Figure 8 by graphing the magnitude of the comb filter using Equation 7 and the Modified Comb Filter code in Appendix A.

$$H_{modified_comb}(e^{j\hat{\omega}}) = \frac{1}{5(3+\sqrt{5})} \left[1 + \left(\frac{\sqrt{5}}{2} + \frac{3}{2}\right)z^{-1} + (\sqrt{5} + 2)z^{-2} + (\sqrt{5} + 3)z^{-3} + (\sqrt{5} + 3)z^{-4} \right. \\ \left. + (\sqrt{5} + 2)z^{-5} + \left(\frac{\sqrt{5}}{2} + \frac{3}{2}\right)z^{-6} + z^{-7} \right] \quad (\text{Eq. 7})$$

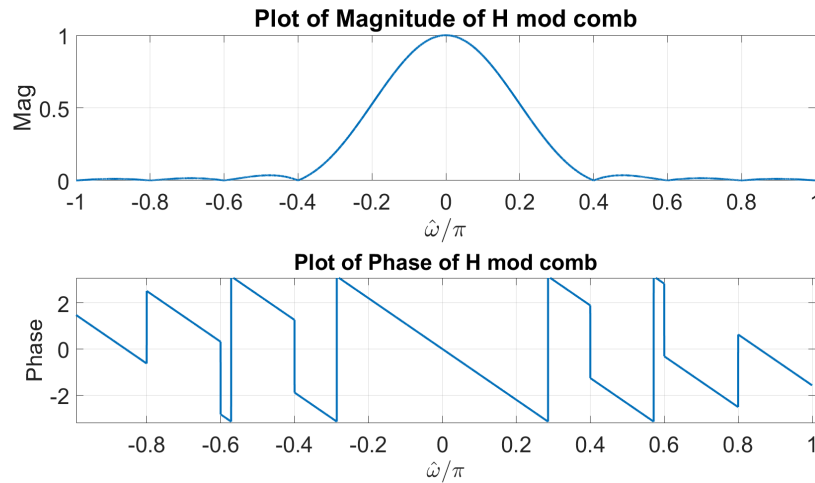


Figure 8: Magnitude and Phase of the Digital Filter

The magnitude of the comb filter were taken at each harmonic value ($0.2 \frac{\widehat{w}}{\pi}$, $0.4 \frac{\widehat{w}}{\pi}$, $0.6 \frac{\widehat{w}}{\pi}$, and $0.8 \frac{\widehat{w}}{\pi}$) and then multiplied by the corresponding input coefficient. This resulted in $|c_{k,estimated}|$, or the estimated output coefficients of the digital filter. The actual values of the coefficients were extracted from the FFT in Figure 9 and recorded in Table 4, along with the ratio between the output and input coefficients.

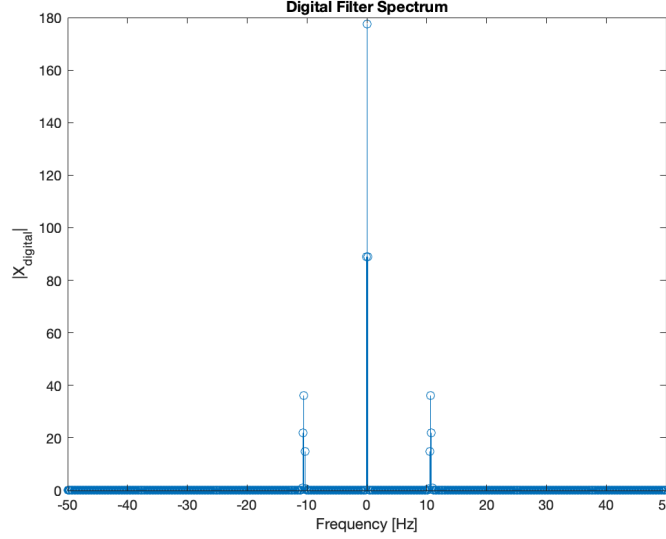


Figure 9: Hanning Window Fourier Spectrum of the Digital Filter

k	$ b_{k,measured} $	$ H(e^{j\widehat{w}}) $	$ c_{k,estimated} $	$ c_{k,measured} $	$\frac{ c_{k,estimated} }{ b_{k,estimated} }$	$\frac{ c_{k,measured} }{ b_{k,measured} }$
1	72.9	0.5	36.45	36.1	1.03	0.50
2	2.78	0.006	0.0167	.0538	0.007	0.019
3	.316	0.002	0.0063	.002	0.026	0.022
4	0.08	0.0016	0.000128	.00397	0.00017	0.1

Table 4: Digital Filter Estimated and Measured Fourier Coefficients

IV. Conclusion

The design of the analog filter depended on a two-stage Sallen-Key configuration to achieve a 4th-order Butterworth filter response. A single Sallen-Key stage produces a second-order filter, so by cascading the two stages the circuit is able to achieve a 4th-order filter. This 4th-order filter is what provides the very flat pass and sharp corner frequency expected from a butterworth filter.

This effectively attenuates the higher frequencies in the signal while maintaining a clean passband.

To address issues of aliasing the sampling frequency was sufficiently greater than twice the highest frequency component of the input signal per the Nyquist theorem. The sampling frequency used by the Arduino was 100 Hz compared to the pulse train frequency of 10 Hz. Frequency resolution in the Fourier analysis is determined by the sampling rate divided by the size of the FFT. The sampling rate was 100 Hz. The size of the FFT for the analog filter was 1235 which results in a sampling resolution of around 80 mHz which is a sufficient resolution for low-frequency signals to be able to distinguish between close harmonics. The FFT size for the digital filter was 655, resulting in a frequency resolution of 0.15 Hz which is still a sufficient resolution given that the bandwidth of interest is 10 Hz.

The application of the Hanning window during the Fourier analysis eliminated the discontinuities from the edges of the sampled signal by smoothing the ends to zero. This results in removing some of the noise from the initial FFT so that the signal is cleaner and more accurate. Applying the Hanning window also results in decreasing the magnitude of the fourier coefficients since some frequencies have been removed. This trade-off is justified by improving the accuracy of the frequency spectrum.

A potential source of error for this system is the differing resistor and capacitor values from the actual components used compared to the values used in the design and derivation for the Sallen-Key filter. Resistor values were calculated to design the filter for a 10 Hz signal and standard resistor values were chosen from those values. The capacitor standard values were set already.

Component	Expected/Standard Value	Measured Value
R_1^1	47 k Ω	46.81 k Ω
R_2^1	270 k Ω	269.9 k Ω
R_1^2	27 k Ω	27.15 k Ω
R_2^2	100 k Ω	100.4 k Ω
C_1^1	0.22 μ F	0.21 μ F

C_2^1	0.1 μF	0.0963 μF
C_1^2	1 μF	0.973 μF
C_2^2	0.1 μF	0.0975 μF

Table 5: Expected and Measured Resistor and Capacitor Values

Comparing the results of the measured and expected Fourier series coefficients of the analog filter shows that the filter performance was as expected. The biggest difference between the estimated and measured coefficient ratios in Table 3 for the analog coefficients was 0.27 when $k = 1$. The values for the remaining harmonics differed by less than 0.008, which is negligible. Looking at the results of the digital filter in Table 4, the biggest difference between the estimated and measured coefficient ratios was around 0.53 for the first harmonic. The values for the other harmonics differed by less than 0.1. Thus the performance of the digital filter was very similar to the expected performance. Reasons for these small discrepancies between the expected and measured values are likely due to the resistor and capacitor values differing from the expected standard values mentioned earlier.

Appendix A: Matlab Code

FFT

```
%FFT.m
load('E101_HW8.mat');
Ts = 0.01;
Fs = 1/Ts;
N = 1000;

X = fft(x)/N;
X = fftshift(X);

f = linspace(-fs/2, fs/2, N);
k = (-500:499);

subplot(2,1,1)
stem(f, abs(X))
title("|X| Spectrum")
xlabel("Frequency [Hz]")
ylabel("|X|")

subplot(2,1,2)
stem(k, abs(X))
xlabel("k")
ylabel("|X|")
```

Hanning Window FFT

```
%Hanning_Window.m
load('E101_HW8.mat');

Ts = 0.01;
Fs = 1/Ts;
N = 1000;
w = hann(N); % hanning window
wX = x(1:N).*w; % apply hanning window to signal

Xhann = fft(wX)/N; % find spectrum of X
Xhann = fftshift(Xhann);

f = linspace(-fs/2, fs/2, N);
k = (-500:499);
```

```

subplot(2,1,1)
stem(f, abs(Xhann))
title("|X_h_a_n_n| Spectrum")
xlabel("Frequency [Hz]")
ylabel("|X_h_a_n_n|")

```

```

subplot(2,1,2)
stem(k,abs(Xhann))
xlabel("k")
ylabel("|X_h_a_n_n|")

```

Sallen Key FRF

```

%Butterworth_Bode.m
wc=2*pi*10; %10Hz as corner frequency fc for final project

```

```

r1 = 47000;
r2 = 270000;
r3 = 27000;
r4 = 100000;
c1 = 2.2*10^(-7);
c2 = 1*10^(-7);
c3 = 1*10^(-6);
c4 = 1*10^(-7);

```

```

H_sallen_key1 = tf(1, [r1*r2*c1*c2 (r1+r2)*c2 1]);
H_sallen_key2 = tf(1, [r3*r4*c3*c4 (r3+r4)*c4 1]);
H_overall = H_sallen_key1*H_sallen_key2;

```

```

figure (1)
opts = bodeoptions;
opts.Title.FontSize = 18;
opts.Title.Color = [0 0 1]; %Blue color
opts.Title.String = '4th order Butterworth Filter FRF';
opts.XLabel.FontSize=12;
opts.YLabel.FontSize=12;
opts.TickLabel.FontSize=12;

```

```

bode(H_overall,opts);
grid on;

```

Modified Comb Filter

```

%Modified_Comb.m
%for modified 8th order comb filter.
b0=1/(5*(3+sqrt(5)));

```

```

delta=0.0001*pi;
w_hat=(-pi:delta:pi); %plot over one period to see the
periodicity.

bb1=conv([1 ((-sqrt(5)/2)+1/2) 1],[1 ((sqrt(5)/2)-1/2) 1]);
bb2=conv(bb1, [1 ((sqrt(5)/2)+1/2) 1]);
b=b0*conv(bb2,[1 1]);

H_modcomb=b(1)+b(2).*exp(-j*w_hat)+b(3).*exp(-j*2*w_hat)+b(4).*e
xp(-j*3*w_hat)+b(5).*exp(-j*4*w_hat)+b(6).*exp(-j*5*w_hat)+b(7).
*exp(-j*6*w_hat)+b(8).*exp(-j*7*w_hat);

f=figure(1);
f.Position=[100 200 800 1000]

subplot(2,1,1);
plot(w_hat/pi,abs(H_modcomb),'LineWidth',2);
ax = gca;
ax.FontSize = 20;
title(['Plot of Magnitude of H mod comb'],'FontSize',22);
xlabel('$\hat{\omega}/\pi$', 'Interpreter','latex','FontSize',22);
ylabel('Mag','FontSize',22);
grid on;
ax = gca;
ax.FontSize = 22;

subplot(2,1,2);
plot(w_hat/pi,angle(H_modcomb),'LineWidth',2);
ax = gca;
ax.FontSize = 22;
title(['Plot of Phase of H mod comb'],'FontSize',22);
xlabel('$\hat{\omega}/\pi$', 'Interpreter','latex','FontSize',22);
ylabel('Phase','FontSize',22);
grid on

figure(2)
stem([0,1,2,3,4,5,6,7],[1*b0, (sqrt(5)/2+3/2)*b0,
(sqrt(5)+2)*b0, (sqrt(5)+3)*b0, (sqrt(5)+3)*b0, (sqrt(5)+2)*b0,
(sqrt(5)/2+3/2)*b0, 1*b0],'LineWidth',2)
title(['Plot of h mod comb'],'FontSize',22);
xlabel('n','FontSize',22);
ylabel('h mod comb','FontSize',22);

```

Fourier Series Calculation for Pulse Train

```
a_0 = 355.3;
A = 1023;
d = a_0/A;
N = 10;
a_1 = abs((A.*sin(1.*pi.*d))/(N.*sin((1.*pi)./N)));
a_2 = abs((A.*sin(2.*pi.*d))/(N.*sin((2.*pi)./N)));
a_3 = abs((A.*sin(3.*pi.*d))/(N.*sin((3.*pi)./N)));
a_4 = abs((A.*sin(4.*pi.*d))/(N.*sin((4.*pi)./N)));
```

Data Processing

```
% Analog Data
fileIDA = fopen('analog10.txt','r');
formatSpec = '%d %f';
sizeA = [2 Inf];
A = fscanf(fileIDA,formatSpec,sizeA)
A = A';
x_analog = [1:length(A)];
y_analog_in = A(:,2)./1000;
y_analog_out = A(:,1)./1000;
% Digital Data
fileIDD= fopen('digital1.txt','r');
formatSpec = '%d %f';
sizeD = [2 Inf];
D = fscanf(fileIDD,formatSpec,sizeD)
D = D';
x_digital = [1:length(D)];
y_digital_in = D(:,2)./1000;
y_digital_out = D(:,1)./1000;
x_color = [0 0.4470 0.7410];
y_color = [0.8500 0.3250 0.0980];
set(0, 'DefaultFigureColor', 'w');
% Plot Analog
figure(1)
plot(x_analog, y_analog_in, "-o", 'Color', x_color, 'DisplayName', 'x[n]'); %
Blue
hold on
plot(x_analog, y_analog_out, "-o", 'Color', y_color, 'DisplayName', 'y[n]'); %
Red
xlim([2 50])
grid on
xlabel("n")
ylabel("[V]")
legend show
title("Analog Filter Input/Output Data")
subtitle("duty cycle = 0.38")
% Plot Digital
hold off
figure(2)
plot(x_digital, y_digital_in, "-o", 'Color', x_color, 'DisplayName', 'x[n]'); %
Blue
hold on
```

```

plot(x_digital, y_digital_out, "-o", 'Color', y_color, 'DisplayName', 'y[n]');
% Red
xlim([2 50])
grid on
xlabel("n")
ylabel("[V]")
legend show
title("Digital Filter Input/Output Data")
subtitle("duty cycle = 0.38")

```

Appendix B: Arduino Code

AnalogInput2.ino

```

/*
Reads 2 analog input pins (range 0-1023) and
prints the results to the serial monitor.

The circuit:
* analog signal connected to analog pin A0.
* analog signal connected to analog pin A1.

*/

// These constants won't change.
// They're used to give names // to the
pins used:
const int inputPin0 = A0; //
Analog input pin 0 const int
inputPin1 = A1; // Analog input
pin 1

void setup() {
  Serial.begin(9600);
}

void loop() {

```

```

    // analogRead values from 0 to
    1023 (10 bit ADC)  int
    sensorVal0 =
    analogRead(inputPin0);
    int sensorVal1 = analogRead(inputPin1);

    // print the results to the
    serial monitor:
    Serial.print(sensorVal0);
    Serial.print(" ");
    Serial.println(sensorVal1);

    delay(10); // sample time ms
}

```

Digital_Filter.ino

```

/*
  Digital FIR Filter

  Reads analog input pin (range 0-1023) and
  prints the filtered output to the serial monitor

  The circuit:
  * analog signal connected to analog pin A1
*/
const int inputPin = A1; // Analog input pin

// filter coefficients
const float b0=0.0382;
const float b1=0.1;
const float b2=0.1618;
const float b3=0.2;
const float b4=0.2;
const float b5=0.1618;
const float b6=0.1;
const float b7=0.0382;

// initial values for internal signals
int xstore1 = 0;

```



```

int xstore2 = 0;
int xstore3 = 0;
int xstore4 = 0;
int xstore5 = 0;
int xstore6 = 0;
int xstore7 = 0;

int yn = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {

  // analogRead value from
  pin A1 (0-1023)  int xn =
  analogRead(inputPin);

  // difference equation for filter
  yn= b0*xn + b1*xstore1 + b2*xstore2 + b3*xstore3 + b4*xstore4 +
  b5*xstore5 + b6*xstore6 + b7*xstore7;

  // print to output
  Serial.print(xn);
  Serial.print(" ");
  Serial.println(yn);

  // update stored values

  xstore7 = xstore6;
  xstore6 = xstore5;
  xstore5 = xstore4;
  xstore4 = xstore3;
  xstore3 = xstore2;
  xstore2 = xstore1;
  xstore1 = xn;

  delay(10); // sample time ms
}

```

Student Team Names: Jared Carreño, Emma Angel, Adne Brako, and Katrina Nguyen

Skills Demonstrated	No ne (0)	Not Ade quate (1)	Mar gin al (2)	Ade quate (3)	Go od (4)	Sup erio r (5)	Mult iplic er	Poin ts
Estimate and measure the Fourier series coefficients of a sampled periodic pulse train signal using windowing of the sampled signal.							3	
Design and build a low pass analog filter using a Sallen-Key design of a 4th order Butterworth Filter.							3	
Estimate and measure the ratios of the Fourier series coefficients of the input and output signals of the analog filter.							3	
Design and code a low pass modified digital comb filter on an Arduino Uno microprocessor.							3	
Estimate and measure the ratio of the Fourier series coefficients of the input and output signals of the digital filter.							3	
Discuss the practical considerations underlying the choice of the op amp stages,							3	

aliasing issues, frequency resolution, and windowing functions.								
Discuss the sources of error between the estimated and measured performance of the filters.							2	
							Final Grade	

Comments:
